

Perico-Perico

Informe tecnico del proyecto

Backend, frontend, tecnologias aplicadas y fragmentos de codigo explicados

Rol	Funciones principales
Pasajero	Busca viajes, reserva asientos, paga, cancela y consulta ubicacion/ruta.
Chofer	Publica viajes, administra autos, cobra, comparte ubicacion y gestiona asientos.
Admin	Gestiona usuarios, datos operativos y permisos ampliados.

1. Arquitectura general

La aplicacion esta separada en dos capas: **PericoBack**, que expone una API REST con Express y guarda datos en PostgreSQL usando Sequelize, y **PericoFront/proyfrontendgrupo03**, que muestra la interfaz Angular para pasajero, chofer y admin.

- **Backend:** Express, Sequelize, PostgreSQL, JWT, Socket.IO, Mercado Pago, Swagger.
- **Frontend:** Angular standalone components, Router, HttpClient, guards, interceptors, Toastr, Leaflet, OSRM, Socket.IO client.
- **Comunicacion:** el front consume endpoints HTTP y recibe actualizaciones en vivo por Socket.IO.

2. Estructura de carpetas

Backend:

- **PericoBack/index.js:** arranque del servidor, CORS, Socket.IO, Swagger y rutas.
- **PericoBack/config/database.js:** conexion Sequelize a PostgreSQL.
- **PericoBack/src/models:** modelos de base de datos.
- **PericoBack/src/models/relaciones.js:** asociaciones entre modelos.
- **PericoBack/src/routes:** rutas HTTP.
- **PericoBack/src/controllers:** logica de negocio.

Frontend:

- **src/app/app.routes.ts:** rutas Angular y roles permitidos.
- **src/app/services:** capa de consumo de API y utilidades.
- **src/app/pages/chofer:** panel del chofer.
- **src/app/pages/pasajero:** panel del pasajero.
- **src/app/components/mapa-ruta:** mapa con Leaflet y ruta OSRM.

3. Autenticacion JWT y roles

El backend protege rutas privadas con JWT. El front guarda el token en sessionStorage y lo envia en el header Authorization. El middleware **verifyToken** valida el token y deja los datos del usuario en **req.usuario**. Despues, **verificarRol** decide si el rol puede usar esa ruta.

```
authCtrl.verifyToken = async (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader) {
    return res.status(401).json({ message: 'Unauthorized request: No token provided.' });
  }

  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.usuario = decoded;
    next();
  } catch (error) {
    return res.status(401).json({ message: 'Unauthorized request: Invalid or expired token.' });
  }
}
```

Explicacion: si el token es valido, el controlador siguiente puede saber quien esta logueado y que rol tiene. Esto evita que un chofer cree viajes para otro chofer o que un pasajero cancele reservas ajenas.

4. Interceptor Angular

El interceptor evita repetir codigo en cada service. Antes de cada request HTTP, busca el token y clona la peticion agregando Authorization: Bearer.

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const loginService = inject(LoginService);
  const token = loginService.obtenerToken();

  if (token) {
    const peticionClonada = req.clone({
      setHeaders: {
        Authorization: `Bearer ${token}`
      }
    });
    return next(peticionClonada);
  }
  return next(req);
};
```

Explicacion: por esto los services no tienen que agregar el JWT manualmente. Si hay sesion, todas las llamadas a la API ya salen autenticadas.

5. Búsqueda de viajes disponibles

El pasajero consulta /api/viajes/disponibles. El backend filtra por origen, destino, estado y cupos. Se permiten viajes ABIERTO y EN_CURSO porque un chofer puede estar trabajando y todavía tener asientos libres.

```
const viajes = await Viaje.findAll({
  where: {
    origen: { [Op.iLike]: origenNormalizado },
    destino: { [Op.iLike]: destinoNormalizado },
    estadoViaje: { [Op.in]: ['ABIERTO', 'EN_CURSO'] },
    asientosDisponibles: { [Op.gt]: 0 }
  },
  include: [
    { association: 'chofer', include: [{ association: 'usuario' }] },
    { association: 'auto' },
    { association: 'reservas' }
  ]
});
```

Explicacion: iLike evita errores por mayusculas/minusculas. Op.in permite varios estados. Op.gt asegura que solo aparezcan viajes con al menos un asiento.

6. Creacion de reservas con transaccion

La reserva y el descuento de asientos se hacen dentro de una transaccion. Esto es importante porque evita que dos pasajeros tomen el ultimo asiento al mismo tiempo.

```
viaje = await Viaje.findByPk(req.body.idViaje, {
  transaction,
  lock: true,
});

if (![ 'ABIERTO', 'EN_CURSO' ].includes(viaje.estadoViaje)) {
  await transaction.rollback();
  return res.status(400).json({ mensaje: 'El viaje no esta disponible para reservas' });
}

if (viaje.asientosDisponibles < cantidadSolicitada) {
  await transaction.rollback();
  return res.status(400).json({ mensaje: 'No hay suficientes asientos disponibles' });
}

reserva = await Reserva.create(req.body, { transaction });
viaje.asientosDisponibles -= cantidadSolicitada;
await viaje.save({ transaction });
await transaction.commit();
```

Explicacion: se bloquea el viaje mientras se valida y actualiza. Si algo falla, rollback deja la base como estaba. Si todo sale bien, commit confirma reserva y cupos.

7. Eventos Socket.IO

Socket.IO mantiene sincronizadas las pantallas. Cuando cambia un viaje, reserva, pago o ubicacion, el backend emite un evento con un nombre que incluye el id afectado.

```
req.io.emit(`asientos_actualizados_viaje_${viaje.idViaje}`, {
  idViaje: viaje.idViaje,
  asientosDisponibles: viaje.asientosDisponibles,
  viaje,
});
```

Explicacion: el id en el nombre del evento evita mezclar actualizaciones entre viajes. El front escucha solo los eventos de las reservas o viajes que esta mostrando.

8. Frontend del pasajero

El componente del pasajero carga su perfil, detecta reserva activa, busca viajes, crea reservas y escucha cambios en vivo.

```
this._pasajeroService.getViajesDisponibles(this.origen, this.destino).subscribe({
  next: (data) => {
    this.viajesDisponibles = data as ViajeCard[];
    this.registrarEventosDeViajesDisponibles();
    this.buscando = false;
    this._changeDetectorRef.detectChanges();
  },
  error: (err) => {
    this.buscando = false;
    this.mensaje = 'No se pudieron cargar los viajes disponibles.';
  }
});
```

Explicacion: el componente no arma la URL de la API; delega eso al PasajeroService. Despues registra eventos Socket.IO para actualizar tarjetas cuando cambian asientos o estado.

9. Frontend del chofer

El componente del chofer carga auto, viaje actual y reservas. Tambien crea viajes, inicia/finaliza, comparte ubicacion y cobra con QR o efectivo.

```
get reservasPendientesPago(): any[] {
  return (this.viajeActual?.reservas || []).filter((reserva) =>
    reserva.estadoPago !== 'PAGADO' &&
    reserva.estadoReserva !== 'CANCELADA' &&
    reserva.estadoReserva !== 'UTILIZADA'
  );
}
```

Explicacion: este getter alimenta la seccion Cobros pendientes. Solo muestra reservas que todavia necesitan cobro y siguen activas.

10. Cobro QR con Mercado Pago

El chofer puede generar un QR para una reserva pendiente. El backend llama a Mercado Pago y tambien convierte qr_data en una imagen base64 para mostrarla en Angular.

```
const qrImage = responseMp.qr_data
  ? await QRCode.toDataURL(responseMp.qr_data)
  : null;

req.io.emit(`qr_generado_reserva_${reserva.idReserva}`, {
  idReserva: reserva.idReserva,
  idViaje: reserva.idViaje,
  qr_data: responseMp.qr_data,
  qr_image: qrImage
});
```

Explicacion: qr_data es la informacion de pago. qr_image permite que el front muestre directamente un QR escaneable sin instalar otra libreria.

11. Webhook de Mercado Pago

Mercado Pago avisa al backend cuando un pago se aprueba. Para link de pago llega como payment; para QR puede llegar como merchant_order. El backend marca la reserva como PAGADO y emite eventos.

```
if (pagoInfo && pagoInfo.status === 'approved') {
  const idReservaLocal = pagoInfo.external_reference;
  const reservaLocal = await Reserva.findByPk(idReservaLocal);

  reservaLocal.estadoPago = 'PAGADO';
  reservaLocal.estadoReserva = 'CONFIRMADA';
  await reservaLocal.save();

  req.io.emit(`pago_confirmado_reserva_${idReservaLocal}`, {
    idReserva: idReservaLocal,
    estadoPago: 'PAGADO',
    estadoReserva: 'CONFIRMADA'
  });
}
```

Explicacion: external_reference conecta el pago de Mercado Pago con la reserva local. El evento permite que pasajero y chofer vean el pago confirmado con toast o actualizacion de pantalla.

12. Mapa, ubicacion y ruta

La ubicacion usa la API nativa del navegador. El mapa usa Leaflet con tiles de OpenStreetMap. Para calcular como llegar, el front consulta OSRM y dibuja el GeoJSON en el mapa.

```
obtenerRuta(origen: CoordenadaMapa, destino: CoordenadaMapa): Observable<any> {
  const coordenadas = `${origen.longitud},${origen.latitud};${destino.longitud},${destino.latitud}`;

  return this._http.get<any>(`${this.osrmUrl}/${coordenadas}`, {
    params: {
      overview: 'full',
      geometries: 'geojson',
      steps: 'true'
    }
  });
}
```

Explicacion: OSRM espera coordenadas como longitud,latitud. La respuesta incluye una geometria GeoJSON que el componente de mapa puede dibujar como linea.

13. Guia rapida para debug

- **No entra a una pagina:** revisar app.routes.ts, auth.guard.ts y el rol guardado en sessionStorage.
- **API devuelve 401/403:** revisar auth.interceptor.ts, JWT y roles en routes.
- **Viaje no aparece:** revisar origen, destino, estado ABIERTO/EN_CURSO y asientosDisponibles.
- **Pago no impacta:** revisar NGROK_URL, webhook /api/reservas/webhook y tokens Mercado Pago.
- **Mapa no muestra ruta:** revisar permisos de ubicacion, RutaService y respuesta de OSRM.

14. Archivos recomendados para estudiar primero

- Frontend: app.routes.ts, login-service.ts, pasajero.service.ts, chofer.service.ts.
- Frontend: pages/pasajero/pasajero.ts y pages/chofer/chofer.ts.
- Backend: routes/viaje.routes.js, routes/reserva.routes.js.
- Backend: controllers/viaje.controller.js, controllers/reserva.controller.js, controllers/auth.controller.js.
- Base de datos: models/relaciones.js.